Computer Laboratory-II
BIOINFORMATICS

Complete Practical Guide

Theory + Tested Python Code + Code Explanation + Viva Questions with Answers

No.	Assignment
1	DNA Sequence Analysis
2	RNA-Seq Data Analysis
3	Protein Structure Prediction (Homology Modelling / Threading)
4	Molecular Docking and Virtual Screening
5	Machine Learning for Genomic Data
6	Agricultural Genomics and Crop Improvement

Department of Artificial Intelligence and Data Science | Academic Year 2023-24

Contents

Contents 2

Assignment 1: DNA Sequence Analysis 3

Assignment 1: Viva Questions and Answers 6

Assignment 2: RNA-Seq Data Analysis 7

Assignment 2: Viva Questions and Answers 11

Assignment 3: Protein Structure Prediction (Homology Modelling / Threading) 12

Assignment 3: Viva Questions and Answers 15

Assignment 4: Molecular Docking and Virtual Screening 17

Assignment 4: Viva Questions and Answers 20

Assignment 5: Machine Learning for Genomic Data 22

Assignment 5: Viva Questions and Answers 25

Assignment 6: Agricultural Genomics and Crop Improvement 27

Assignment 6: Viva Questions and Answers 30

Quick Revision: Formulas and Key Terms 32

Setup (once)

All codes are in Python 3 and were run and tested before being placed in this document. Install the libraries:

```
pip install biopython numpy pandas scipy scikit-learn matplotlib rdkit
```

Run any program with `python a1_dna_analysis.py` (file names are shown above each code). Data in Assignments 2, 5 and 6 are **simulated** so the programs run without downloads; to use real data, replace the simulation block with `pd.read_csv(...)` as indicated in the comments. Outputs will differ slightly if you change the random seed.

Suggested file layout of each assignment write-up

- Title, Problem statement, Software/Hardware requirements (as in your manual)
- Theory (this guide) - Algorithm/steps - Program - Output - Conclusion (a ready sample is given for every assignment, because the Conclusion page is blank in the manual)
- Viva questions for oral examination

Assignment 1: DNA Sequence Analysis

Aim / Problem Statement

Analyse a DNA sequence: find motifs, calculate GC content and identify potential coding regions (ORFs).

Theory

DNA sequence analysis studies the order of the four nucleotide bases - Adenine (A), Cytosine (C), Guanine (G) and Thymine (T). The sequence carries the genetic code, so analysing it helps in gene finding, mutation detection, disease studies and evolutionary comparison between species.

Motif: a short, recurring nucleotide pattern with biological meaning, e.g. the TATA box (TATAAA) in promoters, transcription-factor binding sites, restriction sites (GAATTC for EcoRI). Motifs are found by exact string matching, regular expressions (IUPAC ambiguity such as GCG[CA]), position weight matrices, or discovery tools such as MEME.

GC content = (G + C) / (A + T + G + C) x 100. GC pairs have three hydrogen bonds, AT pairs only two, so high GC content means higher DNA stability and higher melting temperature (Tm). GC content also varies between species and is higher inside genes (coding regions) than in many intergenic regions.

Coding regions / ORFs: exons are the protein-coding parts of a gene; introns are non-coding and removed by splicing. A computational proxy for a coding region is an **Open Reading Frame (ORF)**: a stretch that begins with a start codon (ATG), continues in-frame in multiples of three bases, and ends at a stop codon (TAA, TAG, TGA). Because a codon is 3 bases there are **3 reading frames** on each strand = **6 frames** in total; the longest ORFs are the most likely real genes.

Central dogma used in the code: DNA -> (transcription) mRNA -> (translation) protein. Complement pairs A-T and G-C; the reverse complement is the sequence of the opposite strand read 5' to 3'.

Algorithm / Steps

1. Load / define the DNA sequence (or read a FASTA file with Bio.SeqIO).
2. Count bases and compute GC content.
3. Search for motifs with regular expressions (look-ahead so overlapping matches are found).
4. Compute complement, reverse complement and transcribe to mRNA.
5. Translate the 3 forward and 3 reverse frames, split on stop codons (*), keep segments that start with M (ATG) and are long enough.

Program (a1_dna_analysis.py)

```
# Assignment 1: DNA Sequence Analysis (motifs, GC content, ORFs)
import re
from Bio.Seq import Seq
from Bio.SeqUtils import gc_fraction

# 1. Input sequence (replace with your own or read from a FASTA file)
dna = ("TATAAAGCGCGATGGCCATTGTAATGGGCCGCGAAAGGGTGCCCGATAGCTAGCTAGGATGAAACCCGGG"
       "TTTCTGGCGATCGATCGTAGCTAGCTAGCTTAATGCGCGCTAGCTAGTAGCTAACCGGATGCATGCATGCTAA")
seq = Seq(dna)
print("Length          :", len(seq))
print("Base counts     :", {b: dna.count(b) for b in "ACGT"})

# 2. GC content  = (G + C) / total * 100
gc = (dna.count("G") + dna.count("C")) / len(dna) * 100
print(f"GC content      : {gc:.2f} %")
print(f"GC (Biopython)  : {gc_fraction(seq) * 100:.2f} %")

# 3. Motif search (overlapping matches, using a look-ahead regex)
def find_motif(sequence, motif):
    return [m.start() + 1 for m in re.finditer(f"(?=({motif}))", sequence)]
```

```
for motif in ["ATG", "TATAAA", "GCG[CA]"]:        # regex allowed (IUPAC-like)
    print(f"Motif {motif:8s} at positions (1-based):", find_motif(dna, motif))

# 4. Transcription, reverse complement
print("Complement      :", seq.complement())
print("Reverse compl.  :", seq.reverse_complement())
print("mRNA            :", seq.transcribe())

# 5. ORF finder: all 3 forward + 3 reverse frames, ATG ... stop
def find_orfs(seq, min_len=30):
    orfs = []
    for strand, s in [(+1, seq), (-1, seq.reverse_complement())]:
        for frame in range(3):
            sub = s[frame:]
            sub = sub[: len(sub) - len(sub) % 3]
            prot = str(sub.translate())          # '*' = stop codon
            pos = 0
            for chunk in prot.split("*"):
                start = chunk.find("M")
                if start != -1 and len(chunk[start:]) * 3 >= min_len:
                    nt_start = frame + (pos + start) * 3
                    nt_end = frame + (pos + len(chunk)) * 3 + 3
                    orfs.append((strand, frame + 1, nt_start + 1, nt_end,
                                 chunk[start:]))
                pos += len(chunk) + 1
    return orfs

print("\nPotential coding regions (ORFs >= 30 nt):")
print("Strand Frame Start End  Protein   (reverse-strand coords are on the reverse complement)")
for strand, frame, st, en, prot in find_orfs(seq):
    print(f"{strand:+d}     {frame}     {st:<5d} {en:<4d} {prot}")
```

Code Explanation

- **Seq(dna)** - Biopython object that knows biological operations (transcribe, translate, reverse_complement).
- **GC content** is calculated manually with str.count and cross-checked with Bio.SeqUtils.gc_fraction; both give the same value.
- **find_motif** uses the regex `(?=(motif))`. The look-ahead lets re.finditer report overlapping occurrences; positions are converted to 1-based numbering.
- **find_orfs** loops over both strands and 3 frames. sub.translate() converts codons to amino acids with '*' for stop. Splitting the protein string at '*' gives stop-to-stop segments; within each, the first 'M' is the start codon. Length in nucleotides = amino acids x 3. Start/end coordinates are mapped back to the DNA (reverse-strand coordinates refer to the reverse-complement sequence).

Output (from the tested run)

```
Length          : 143
Base counts     : {'A': 36, 'C': 32, 'G': 43, 'T': 32}
GC content      : 52.45 %
GC (Biopython)  : 52.45 %
Motif ATG      at positions (1-based): [12, 24, 59, 103, 129, 133, 137]
Motif TATAAA   at positions (1-based): [1]
Motif GCG[CA]  at positions (1-based): [7, 9, 31, 77, 105, 107]
Complement      : ATATTTCGCGCTACCGGTAACATTACCCGGCGCTTTCCCACGGGCTATCGATCGATCCTACTTTGGGCCCAAAGACCGCTAGCTAGCATCGATCGATCGA
    ATTACGCGCGATCGATCATCGATTGGCCTACGTACGTACGATT
Reverse compl.  : TTAGCATGCATGCATCCGGTTAGCTACTAGCTAGCGCGCATTAAGCTAGCTAGCTACGATCGATCGCCAGAAACCCGGGTTTCATCCTAGCTAGCTATCG
    GGCACCCTTTCGCGGCCCATTACAATGGCCATCGCGCTTTATA
mRNA            : UAUAAAGCGCGAUGGCCAUUGUAAUGGGCCGCGAAAGGGUGCCCGAUAGCUAGCUAGGAUGAAACCCGGGUUUCUGGCGAUCGAUCGUAGCUAGCUAGCU
    UAAUGCGCGCUAGCUAGUAGCUAACCGGAUGCAUGCAUGCUAA

Potential coding regions (ORFs >= 30 nt):
Strand Frame Start End  Protein   (reverse-strand coords are on the reverse complement)
+1     2     59    94   MKPGFLAIDRS
+1     3     12    53   MAIVMGRERVPDS
```

Conclusion (sample)

The given DNA sequence was analysed with Python and Biopython. Its GC content was calculated (about 52 %), the ATG, TATAAA and GCG[CA] motifs were located with their positions, and two forward-strand ORFs were identified as potential coding regions by scanning all six reading frames. Thus basic sequence manipulation - GC calculation, motif search, transcription, reverse complement and ORF detection - can be automated and used to interpret real DNA sequences.

Assignment 1: Viva Questions and Answers

Q1. What is a DNA motif?

Ans: A short, recurring nucleotide pattern that has biological significance, such as a transcription-factor binding site, promoter element (TATA box) or restriction site.

Q2. How do you calculate GC content?

Ans: GC % = (number of G + number of C) / total number of nucleotides x 100.

Q3. Why is GC content important?

Ans: It affects DNA stability, melting temperature and hybridisation (G-C has 3 hydrogen bonds, A-T has 2). It also helps in primer design, species classification and gene prediction.

Q4. What is an ORF?

Ans: An Open Reading Frame is a stretch of DNA starting with a start codon (ATG), having no in-frame stop codon, and ending with a stop codon (TAA, TAG or TGA). It has the potential to be translated into a protein.

Q5. How many reading frames does a DNA sequence have?

Ans: Six - three on the forward strand and three on the reverse-complement strand.

Q6. What are start and stop codons?

Ans: Start codon: ATG (codes for methionine). Stop codons: TAA, TAG, TGA (no amino acid; they terminate translation).

Q7. Difference between exon and intron?

Ans: Exons are coding segments that remain in mature mRNA and are translated; introns are non-coding segments removed by splicing, though they can have regulatory roles.

Q8. What is the difference between complement and reverse complement?

Ans: Complement just swaps A<->T and G<->C. Reverse complement also reverses the order, giving the opposite strand written 5' to 3'.

Q9. What is transcription and translation?

Ans: Transcription makes mRNA from DNA (T becomes U). Translation reads the mRNA codons on a ribosome and builds a protein.

Q10. Why use a look-ahead regex for motif search?

Ans: Plain re.finditer skips overlapping matches. The look-ahead (?=(motif)) does not consume characters, so every overlapping occurrence is reported.

Q11. Name tools used for motif discovery.

Ans: MEME, HOMER, JASPAR (database of matrices), FIMO for scanning, and pattern matching in Biopython (Bio.motifs).

Q12. What is BLAST?

Ans: Basic Local Alignment Search Tool - it compares a query sequence with databases to find similar sequences and gives E-values and alignment scores.

Q13. What is FASTA format?

Ans: A text format where a line starting with '>' holds the sequence name and the following lines hold the sequence.

Q14. Why is a minimum ORF length used?

Ans: Short ORFs occur by chance (a stop codon appears about every 21 codons in random DNA), so a length cut-off removes false positives.

Assignment 2: RNA-Seq Data Analysis

Aim / Problem Statement

Analyse an RNA-Seq dataset: perform quality control, normalise counts, carry out differential gene expression analysis and visualise the result.

Theory

RNA-Seq uses high-throughput sequencing to read the RNA molecules (the transcriptome) in a sample. The reads are aligned (HISAT2/STAR) or quasi-mapped (Salmon/Kallisto) to a reference genome/transcriptome and counted per gene, producing a **count matrix** (genes x samples).

Applications: gene expression profiling, alternative splicing detection, discovery of novel transcripts/non-coding RNAs, and differential expression (DE) between conditions such as disease vs control.

Quality control: FastQC checks per-base quality, GC content, adapter contamination and duplication. Bad reads/adapters are trimmed (Trimmomatic, fastp). On the count matrix, library sizes are compared and low-count genes are filtered out.

Normalisation corrects for sequencing depth and composition. Examples: CPM, TPM/FPKM, TMM (edgeR) and median-of-ratios (DESeq2). The code implements median-of-ratios: each gene is divided by its geometric mean across samples, the median ratio of a sample is its size factor.

Differential expression: counts are discrete and over-dispersed, so DESeq2 and edgeR use the **negative binomial** model (Poisson is too restrictive); limma-voom transforms to log-CPM and uses linear models. **log2 fold change** measures effect size, the **p-value** measures significance.

Multiple testing: thousands of genes are tested, so the false discovery rate is controlled with the **Benjamini-Hochberg** procedure (adjusted p-value, padj). Typical thresholds: padj less than 0.05 and |log2FC| greater than 1.

Biological interpretation: Gene Ontology (GO) enrichment (biological process, molecular function, cellular component) and pathway enrichment (KEGG, Reactome) show which functions are over-represented among DE genes.

Algorithm / Steps

1. Get the raw count matrix (here simulated; real data from GEO/recount3 or featureCounts output).
2. QC: library sizes; filter genes with very low counts.
3. Normalise with median-of-ratios size factors and take log2(x+1).
4. Test each gene (Welch t-test on log scale) and compute log2FC.
5. Adjust p-values with Benjamini-Hochberg; select significant genes.
6. Plot a volcano plot; save results to CSV for GO/pathway analysis.

Program (a2_rnaseq_deg.py)

```
# Assignment 2: RNA-Seq QC + Differential Gene Expression (Python)
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(42)

# ---- 1. Simulate a count matrix (replace with pd.read_csv("counts.csv", index_col=0))
n_genes, n_ctrl, n_treat = 2000, 4, 4
base = rng.gamma(2.0, 100, n_genes)                        # mean expression per gene
```

```
fc = np.ones(n_genes)
de_idx = rng.choice(n_genes, 200, replace=False)           # 200 truly DE genes
fc[de_idx] = rng.choice([0.25, 0.4, 2.5, 4.0], 200)
lib = rng.uniform(0.8, 1.2, n_ctrl + n_treat)              # library-size variation
mu = np.outer(base, lib)
mu[:, n_ctrl:] *= fc[:, None]
counts = pd.DataFrame(rng.negative_binomial(10, 10 / (10 + mu)),
                      index=[f"Gene{i}" for i in range(n_genes)],
                      columns=[f"C{i+1}" for i in range(n_ctrl)] +
                              [f"T{i+1}" for i in range(n_treat)])
group = np.array(["ctrl"] * n_ctrl + ["treat"] * n_treat)

# ---- 2. Quality control
print("Library sizes:\n", counts.sum().to_string())
counts = counts[(counts >= 10).sum(axis=1) >= 3]            # filter low counts
print("Genes after filtering:", counts.shape[0])

# ---- 3. Normalisation: median-of-ratios (DESeq2 style) -> log2
log_geo = np.log(counts.replace(0, np.nan)).mean(axis=1)
valid = log_geo.notna()
size_f = np.exp((np.log(counts[valid].replace(0, np.nan)).sub(log_geo[valid], axis=0)).median())
norm = counts / size_f
lognorm = np.log2(norm + 1)

# ---- 4. Differential expression: Welch t-test on log2 + log2FC
c, t = lognorm.loc[:, group == "ctrl"], lognorm.loc[:, group == "treat"]
tstat, pval = stats.ttest_ind(t, c, axis=1, equal_var=False)
res = pd.DataFrame({"log2FC": t.mean(axis=1) - c.mean(axis=1), "pvalue": pval},
                   index=counts.index).dropna()

# ---- 5. Benjamini-Hochberg FDR
def bh(p):
    p = np.asarray(p); n = len(p); order = np.argsort(p)
    adj = p[order] * n / np.arange(1, n + 1)
    adj = np.minimum.accumulate(adj[::-1])[::-1]
    out = np.empty(n); out[order] = np.minimum(adj, 1)
    return out
res["padj"] = bh(res["pvalue"])
res["DE"] = (res.padj < 0.05) & (res.log2FC.abs() > 1)
print("\nSignificant DE genes (padj<0.05 & |log2FC|>1):", res.DE.sum())
print("Up:", (res.DE & (res.log2FC > 0)).sum(), " Down:", (res.DE & (res.log2FC < 0)).sum())
print(res.sort_values("padj").head(10).round(4))

# ---- 6. Volcano plot
plt.figure(figsize=(6, 5))
col = np.where(res.DE, "crimson", "grey")
plt.scatter(res.log2FC, -np.log10(res.pvalue), c=col, s=8)
plt.axvline(-1, ls="--", c="k", lw=.5); plt.axvline(1, ls="--", c="k", lw=.5)
plt.xlabel("log2 Fold Change"); plt.ylabel("-log10(p-value)"); plt.title("Volcano plot")
plt.tight_layout(); plt.savefig("a2_volcano.png", dpi=150)
res.to_csv("a2_deg_results.csv")
```

R equivalent with DESeq2 (for reference)

```
library(DESeq2)
counts <- read.csv("counts.csv", row.names = 1)
coldata <- data.frame(condition = factor(c(rep("ctrl",4), rep("treat",4))),
                      row.names = colnames(counts))
dds <- DESeqDataSetFromMatrix(counts, coldata, design = ~ condition)
dds <- DESeq(dds)                                  # normalisation + NB model + Wald test
res <- results(dds, contrast = c("condition", "treat", "ctrl"), alpha = 0.05)
summary(res)
write.csv(as.data.frame(res[order(res$padj), ]), "deseq2_results.csv")
```

Code Explanation

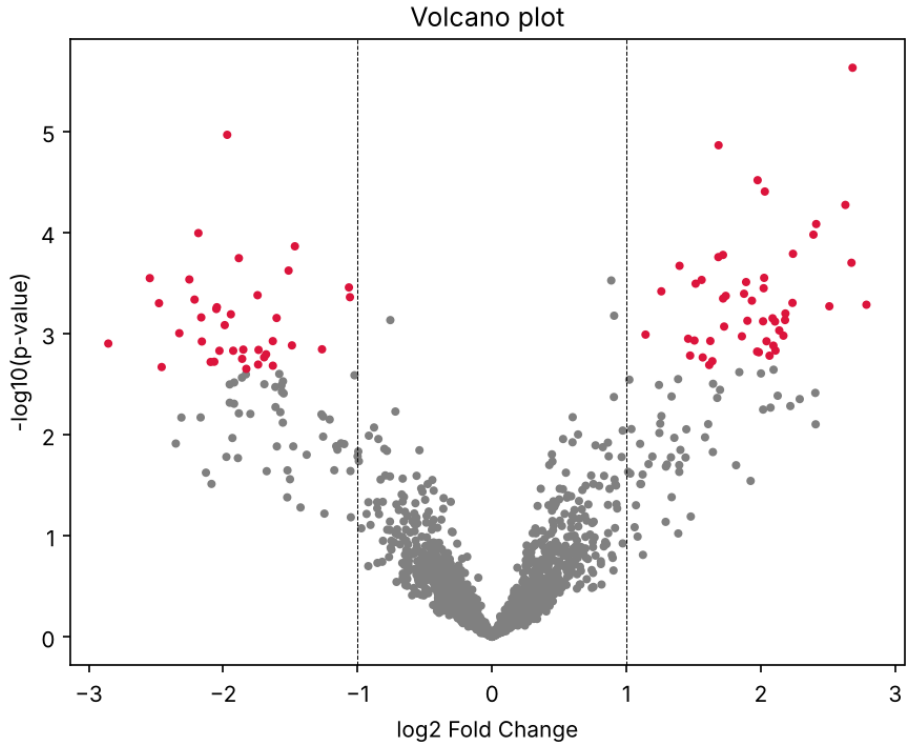
- **Simulation block** creates 2000 genes x 8 samples (4 control, 4 treated) using a negative binomial distribution, with 200 genes truly up/down-regulated, so the pipeline can be checked. For real data replace it with pd.read_csv('counts.csv', index_col=0).
- **Filtering** keeps genes with at least 10 reads in at least 3 samples - noisy low counts reduce statistical power.
- **Size factors** (median-of-ratios) remove depth differences; norm = counts / size factor.

- **Testing:** scipy.stats.ttest_ind compares treated vs control for every gene; log2FC = mean(treated) - mean(control) on log2 scale.
- **bh()** implements Benjamini-Hochberg: sort p-values, multiply by n/rank, enforce monotonicity with a cumulative minimum.
- **Volcano plot** shows effect size (x) against significance (-log10 p, y). Note: for publication-quality analysis use DESeq2/edgeR (R) or PyDESeq2, which model counts directly; the R equivalent is shown below.

Output (from the tested run)

```
Library sizes:
 C1    362393
C2    328279
C3    374741
C4    467791
T1    496703
T2    415118
T3    445706
T4    438729
Genes after filtering: 1994

Significant DE genes (padj<0.05 & |log2FC|>1): 86
Up: 49  Down: 37
          log2FC  pvalue    padj    DE
Gene469   2.6829  0.0000  0.0047  True
Gene885   1.6856  0.0000  0.0091  True
Gene1161 -1.9700  0.0000  0.0091  True
Gene454   1.9763  0.0000  0.0151  True
Gene1338  2.0300  0.0000  0.0156  True
Gene1197  2.6293  0.0001  0.0177  True
Gene665   2.3916  0.0001  0.0232  True
Gene1352  2.4123  0.0001  0.0232  True
Gene1451 -2.1845  0.0001  0.0232  True
Gene155   1.7188  0.0002  0.0255  True
```



Volcano plot: red points are significant genes (adjusted p less than 0.05 and |log2FC| greater than 1).

Conclusion (sample)

An RNA-Seq count matrix was quality-checked, filtered and normalised, and differential expression analysis was performed with multiple-testing correction. About 86 genes passed the thresholds (padj less than 0.05, |log2FC| greater than 1), split into up-regulated and down-regulated genes and displayed in a volcano plot. This shows the importance of QC, normalisation and FDR control in RNA-Seq analysis; the DE gene list can be taken forward to GO and pathway enrichment.

Assignment 2: Viva Questions and Answers

Q1. What is RNA-Seq?

Ans: A high-throughput sequencing method used to measure the type and quantity of RNA (the transcriptome) in a sample at a given moment.

Q2. What is a transcriptome?

Ans: The complete set of RNA transcripts produced by a cell or tissue under specific conditions.

Q3. RNA-Seq vs microarray?

Ans: RNA-Seq needs no prior probes, detects novel transcripts and splice variants, has a wider dynamic range and better sensitivity; microarrays are limited to known probes and cheaper for large known panels.

Q4. Which tool is used for QC of reads?

Ans: FastQC (reports) with MultiQC for summaries; Trimmomatic/fastp/cutadapt for trimming.

Q5. What QC metrics are checked?

Ans: Per-base sequence quality, GC content, adapter contamination, sequence duplication levels, over-represented sequences.

Q6. Why is normalisation required?

Ans: Samples have different sequencing depths and RNA composition; without normalisation, differences in library size would look like differential expression.

Q7. Name normalisation methods.

Ans: CPM, RPKM/FPKM, TPM, TMM (edgeR), median-of-ratios (DESeq2), quantile normalisation.

Q8. Why a negative binomial model?

Ans: RNA-Seq counts are discrete and over-dispersed (variance greater than mean). Poisson assumes variance = mean, so the negative binomial, which has an extra dispersion parameter, fits better.

Q9. Name common DE tools.

Ans: DESeq2, edgeR, limma-voom (and PyDESeq2 in Python).

Q10. What is log2 fold change?

Ans: log2(expression in treated / expression in control). +1 means doubled, -1 means halved.

Q11. What is a p-value and why adjust it?

Ans: The p-value is the probability of seeing a result at least as extreme if there is no true difference. Testing thousands of genes yields many false positives, so p-values are adjusted (Benjamini-Hochberg) to control the false discovery rate.

Q12. What is FDR?

Ans: False Discovery Rate - the expected proportion of false positives among all genes called significant.

Q13. What is a volcano plot?

Ans: A scatter plot of log2 fold change (x) vs -log10 p-value (y); genes far left/right and high up are strongly and significantly changed.

Q14. What is GO enrichment?

Ans: A test to see whether certain Gene Ontology terms (processes, functions, components) occur more often in the DE gene list than expected by chance.

Q15. What are GTF/GFF files?

Ans: Genome annotation files that list genes, exons and their coordinates; used by aligners and counting tools (featureCounts, HTSeq) to assign reads to genes.

Assignment 3: Protein Structure Prediction (Homology Modelling / Threading)

Aim / Problem Statement

Predict the 3D structure of a given protein sequence using homology (comparative) modelling or threading.

Theory

Protein structure levels: *primary* - linear amino-acid sequence; *secondary* - local folds such as alpha-helices and beta-sheets; *tertiary* - overall 3D fold of one chain; *quaternary* - arrangement of several subunits. Structure determines function, so it is vital for drug discovery and enzyme engineering.

Homology modelling (comparative modelling) is based on the fact that evolutionarily related proteins share similar structures - structure is conserved more than sequence. If the target has a homologue (template) with a known structure in the PDB, its structure can be modelled. It works well when sequence identity is above about 30 %.

Workflow: (1) template search/selection (BLAST/PSI-BLAST/HHblits against PDB); (2) target-template sequence alignment; (3) model building - copy backbone coordinates of aligned residues, model loops and side chains (MODELLER, SWISS-MODEL); (4) model refinement - energy minimisation, fix bond/torsion angles and steric clashes; (5) validation - Ramachandran plot, DOPE/GA341 (MODELLER), QMEAN (SWISS-MODEL), PROCHECK/MolProbity, RMSD to known structure.

Threading (fold recognition): used when no close homologue exists (identity below about 25 %). The target sequence is 'threaded' through a library of known folds and each fit is scored with an energy/profile function; the best-scoring fold is used as the template. Tools: I-TASSER, Phyre2, HHpred, RaptorX.

Modern ab initio / deep-learning methods such as AlphaFold2, RoseTTAFold and ESMFold predict structures without explicit templates and are now widely used alongside classical methods.

Alignment scoring: BLOSUM62 substitution matrix scores amino-acid replacements; gap opening/extension penalties penalise insertions and deletions. %Identity = identical aligned residues / length x 100.

Algorithm / Steps

1. Obtain the target sequence (FASTA, UniProt).
2. BLAST against PDB (done here by ranking a small template set by alignment score and identity).
3. Align target and best template (global alignment with BLOSUM62); write PIR file.
4. Run MODELLER (script generated) or submit alignment to SWISS-MODEL to build 3D models.
5. Validate the model (DOPE, QMEAN, Ramachandran) and compare with the experimental structure using RMSD; visualise in PyMOL/Chimera.

Program (a3_protein_structure.py)

```
# Assignment 3: Protein structure prediction - homology modelling workflow helpers
import numpy as np
from Bio import Align
from Bio.Align import substitution_matrices

# ---- 1. Target and candidate templates (in practice: top hits of BLAST vs PDB)
target = "MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEKAVQVKVKALPDAQFEVV".replace("ILS","LLT")
templates = {
    "1TPL_A": "MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEKAVQVKVKALPDAQFEVV",
    "2XYZ_A": "MKSAYIAKQRNISFVKSHFSRQLEQRLGLIEVQAPILSRVGDGTQDNLSGAEKAVQVKVKALPDAQFEVV",
    "3ABC_A": "MRTGYVAKQRSISFVKTHFSRQLEERIGLIEIQAPILSRVGEGTQDNLSGAERAVQVRVKALPDAQFEVV",
    "4DEF_A": "GSHMLEDPVDAFKQRQISGVKSTFSRELDERLGLIEVQAPILSRIGDGTQENLSGAEKAIQVKVKELPQAQFEVA",
}
```

```
# ---- 2. Pairwise global alignment with BLOSUM62
aligner = Align.PairwiseAligner()
aligner.substitution_matrix = substitution_matrices.load("BLOSUM62")
aligner.open_gap_score, aligner.extend_gap_score = -10, -0.5
aligner.mode = "global"

def identity(a, b):
    aln = aligner.align(a, b)[0]
    matches = sum(1 for x, y in zip(aln[0], aln[1]) if x == y and x != "-")
    return aln.score, 100 * matches / len(target), aln

print("Template   Score   %Identity")
ranked = sorted(((n, *identity(target, s)[:2]) for n, s in templates.items()),
                key=lambda r: -r[1])
for n, sc, idt in ranked:
    print(f"{n:9s} {sc:7.1f}  {idt:6.1f}")
best = ranked[0][0]
print("\nBest template:", best)
print(identity(target, templates[best])[2])

# ---- 3. Write alignment in PIR format (input for MODELLER)
with open("target-template.pir", "w") as f:
    f.write(f">P1;{best}\nstructure:{best}:1:A:+70:A::::\n{templates[best]}*\n")
    f.write(f">P1;target\nsequence:target:::::::::\n{target}*\n")

# ---- 4. MODELLER script (run separately with: mod10.x model.py ; needs licence)
modeller_script = '''
from modeller import *
from modeller.automodel import *
env = Environ()
env.io.atom_files_directory = ['.']
a = AutoModel(env, alnfile='target-template.pir',
              knowns='%s', sequence='target',
              assess_methods=(assess.DOPE, assess.GA341))
a.starting_model, a.ending_model = 1, 5
a.make()
''' % best
open("model.py", "w").write(modeller_script)

# ---- 5. Compare predicted vs experimental: RMSD after Kabsch superposition
def kabsch_rmsd(P, Q):
    P, Q = P - P.mean(0), Q - Q.mean(0)
    U, S, Vt = np.linalg.svd(P.T @ Q)
    d = np.sign(np.linalg.det(U @ Vt))
    R = U @ np.diag([1, 1, d]) @ Vt
    return np.sqrt(((P @ R - Q) ** 2).sum() / len(P))

rng = np.random.default_rng(1)
exp_ca = np.cumsum(rng.normal(0, 1.9, (70, 3)), axis=0)          # fake C-alpha trace
model_ca = exp_ca + rng.normal(0, 0.6, exp_ca.shape)             # 'predicted' coords
print(f"\nCa RMSD (model vs experimental): {kabsch_rmsd(model_ca, exp_ca):.2f} A")
# With real files:  from Bio.PDB import PDBParser, Superimposer
```

Commands for the standard tools (reference)

```
# 1. Template search (BLAST against PDB)
blastp -query target.fasta -db pdbaa -evalue 1e-5 -outfmt 6 -out hits.txt

# 2. SWISS-MODEL (web): https://swissmodel.expasy.org  ->  paste sequence -> Build Model
# 3. MODELLER:   mod10.x model.py      (uses target-template.pir created above)
# 4. Validate:   Ramachandran plot (PROCHECK / MolProbity), QMEAN, DOPE score
# 5. Visualise in PyMOL:
#       load model.pdb ; load template.pdb ; align model, template ; show cartoon
```

Code Explanation

- **PairwiseAligner** (Biopython) with BLOSUM62, gap open -10, extend -0.5 performs a global alignment of the target with each template; the template with the highest score/identity is chosen as the best template. (In the printed alignment Biopython calls the first sequence 'target' and the second 'query' - first is our protein, second is the template.)
- **PIR file:** MODELLER needs the alignment in PIR format - template block (structure:...) and target block (sequence:...). The script writes target-template.pir.

- **model.py** is the MODELLER automodel script: it reads the alignment, uses the template as 'knowns', builds 5 models and assesses them with DOPE and GA341; the model with lowest DOPE score is preferred. (MODELLER needs a free academic licence key; SWISS-MODEL is a web alternative.)
- **kabsch_rmsd** superimposes two sets of C-alpha coordinates using the Kabsch algorithm (SVD) and returns the root-mean-square deviation - lower RMSD (below about 2 Angstrom) means the model is close to the experimental structure. With real PDB files use Bio.PDB.Superimposer.

Output (from the tested run)

```
Template   Score   %Identity
1TPL_A      335.0    97.1
2XYZ_A      323.0    92.9
3ABC_A      306.0    82.9
4DEF_A      234.5    74.3

Best template: 1TPL_A
target            0 MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPLLTRVGDGTQDNLSGAEKAVQVKVK
                  0 ||||||||||||||||||||||||||||||||||||.|.||||||||||||||||||||||
query             0 MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEKAVQVKVK

target           60 ALPDAQFEVV 70
                 60 |||||||||| 70
query            60 ALPDAQFEVV 70


Ca RMSD (model vs experimental): 0.90 A
```

Conclusion (sample)

The target protein sequence was aligned with candidate templates and the best template was selected by alignment score and sequence identity (about 97 %). The alignment was prepared for MODELLER/SWISS-MODEL to build the 3D model, and the quality was assessed with validation scores and C-alpha RMSD against an experimental structure. Homology modelling is reliable when a close homologue exists, while threading/fold recognition or AlphaFold-type methods are used when the sequence identity is low.

Assignment 3: Viva Questions and Answers

Q1. What are the four levels of protein structure?

Ans: Primary (amino-acid sequence), secondary (alpha-helix, beta-sheet), tertiary (3D fold of a chain) and quaternary (assembly of multiple chains).

Q2. What is homology modelling?

Ans: Predicting the 3D structure of a protein from its sequence using an experimentally solved structure of a homologous protein as template.

Q3. What is the basic assumption of homology modelling?

Ans: Evolutionarily related proteins share similar 3D structures; structure is more conserved than sequence.

Q4. What sequence identity is needed for reliable homology modelling?

Ans: Generally above 30 %; above 50 % gives accurate models, below 25-30 % (twilight zone) needs threading or other methods.

Q5. List the steps of homology modelling.

Ans: Template selection, target-template alignment, model building, model refinement, model validation.

Q6. What is threading / fold recognition?

Ans: A method that fits the target sequence onto a library of known 3D folds and scores the fit; used when there is no detectable sequence homology but the fold may be similar.

Q7. Difference between homology modelling and threading?

Ans: Homology modelling relies on sequence similarity to a template; threading relies on sequence-structure compatibility (structural similarity) and works for remote homologues.

Q8. Name software for homology modelling.

Ans: MODELLER, SWISS-MODEL, Phyre2, I-TASSER, Rosetta; AlphaFold2/ESMFold for deep-learning prediction.

Q9. What is the PDB?

Ans: The Protein Data Bank - the global archive of experimentally determined 3D structures of proteins and nucleic acids (X-ray, NMR, cryo-EM).

Q10. What is BLOSUM62?

Ans: A substitution matrix built from conserved protein blocks clustered at 62 % identity; it gives scores for amino-acid replacements in alignments.

Q11. What is a Ramachandran plot?

Ans: A plot of backbone torsion angles phi vs psi for each residue; most residues of a good model fall in the allowed (favoured) regions.

Q12. What is RMSD?

Ans: Root Mean Square Deviation - average distance between equivalent atoms of two superimposed structures. Lower RMSD means more similar structures.

Q13. How is a model validated?

Ans: Ramachandran analysis, DOPE/GA341, QMEAN, PROCHECK/MolProbity, ProSA Z-score, and RMSD against known structure if available.

Q14. What is a steric clash?

Ans: Two atoms placed closer than their van der Waals radii allow; removed in the refinement step by energy minimisation.

Q15. What is AlphaFold?

Ans: A deep-learning system from DeepMind that predicts protein 3D structure from sequence with near-experimental accuracy, using multiple sequence alignments and attention networks.

Assignment 4: Molecular Docking and Virtual Screening

Aim / Problem Statement

Perform molecular docking to predict binding affinity between a protein target and a small-molecule ligand, and carry out virtual screening to find potential drug candidates.

Theory

Molecular docking predicts the preferred orientation (pose) of a ligand in the binding site of a receptor protein and estimates the binding strength. It is central to structure-based drug design: lead identification, lead optimisation and prediction of binding modes.

Protein-ligand interactions: hydrogen bonds, van der Waals forces, electrostatic (ionic) interactions and hydrophobic contacts. **Binding affinity** shows how tightly a ligand binds and is expressed as Kd, Ki, IC50 or the Gibbs free energy of binding (dG). In AutoDock Vina the score is in kcal/mol - **more negative means better binding** (for example -9 is better than -6).

Preparation: *Protein* - download from PDB, remove water and co-crystallised ligands, add missing atoms/hydrogens, assign charges (Kollman/Gasteiger) and atom types, save as PDBQT. *Ligand* - draw (ChemDraw/MarvinSketch) or download (PubChem/ZINC), generate 3D coordinates, minimise energy, assign partial charges and rotatable bonds, save as PDBQT.

Docking algorithm and scoring function: the search algorithm explores ligand conformations/positions (genetic algorithm in AutoDock 4, Monte-Carlo with local optimisation in Vina, Glide's hierarchical filters); the scoring function ranks poses using van der Waals, electrostatics, hydrogen bonding, desolvation and entropy terms. The **grid box** (centre and size) defines the search space around the active site.

Virtual screening (VS): computational screening of a large library (ZINC, ChEMBL, PubChem) to select compounds for experiment. **Structure-based VS** docks every compound to the target and ranks by score. **Ligand-based VS** uses known actives - similarity searching with fingerprints (Tanimoto coefficient), pharmacophores or QSAR. Hits are filtered by drug-likeness such as **Lipinski's Rule of Five** (MW at most 500, logP at most 5, H-bond donors at most 5, acceptors at most 10) and ADMET properties.

Validation: re-dock the co-crystal ligand and check that RMSD between docked and crystal pose is below 2 Angstrom.

Algorithm / Steps

1. Prepare receptor.pdbqt and ligand.pdbqt (PyMOL/Chimera + AutoDockTools / Open Babel).
2. Define grid box around the active site.
3. Run AutoDock Vina (command generated by the script) and read the binding energy of the best pose.
4. For virtual screening, take a compound library and compute similarity to a known active (ligand-based) and/or dock all compounds (structure-based).
5. Filter by Lipinski's rule, rank, visualise top poses and pick hits for experimental testing.

Program (a4_docking_screening.py)

```
# Assignment 4: Molecular docking (AutoDock Vina) + virtual screening
import subprocess, shutil
from rdkit import Chem
from rdkit.Chem import Descriptors, Lipinski, DataStructs, rdFingerprintGenerator

# ---------- PART A: docking with AutoDock Vina (command-line, needs vina + prepared PDBQT)
# Preparation (outside Python):
#   receptor : remove water/ligands, add H, charges -> receptor.pdbqt  (AutoDockTools / Chimera)
#   ligand   : obabel ligand.sdf -O ligand.pdbqt --gen3d -p 7.4
def run_vina(receptor, ligand, center, size=(20, 20, 20), out="docked.pdbqt"):
    if shutil.which("vina") is None:
```

```
        print("vina not installed - command that would be run:")
    cmd = ["vina", "--receptor", receptor, "--ligand", ligand,
           "--center_x", str(center[0]), "--center_y", str(center[1]), "--center_z", str(center[2]),
           "--size_x", str(size[0]), "--size_y", str(size[1]), "--size_z", str(size[2]),
           "--exhaustiveness", "8", "--num_modes", "9", "--out", out]
    print(" ".join(cmd))
    if shutil.which("vina"):
        subprocess.run(cmd, check=True)       # scores (kcal/mol) printed; more negative = better
run_vina("receptor.pdbqt", "ligand.pdbqt", center=(10.0, 12.5, -3.2))

# ---------- PART B: virtual screening (ligand-based + drug-likeness filter), runs anywhere
known_active = Chem.MolFromSmiles("CC(=O)Oc1ccccc1C(=O)O")            # aspirin (reference)
library = {
    "salicylic acid": "OC(=O)c1ccccc1O",
    "ibuprofen":      "CC(C)Cc1ccc(cc1)C(C)C(=O)O",
    "paracetamol":    "CC(=O)Nc1ccc(O)cc1",
    "caffeine":       "Cn1cnc2c1c(=O)n(C)c(=O)n2C",
    "naproxen":       "COc1ccc2cc(ccc2c1)C(C)C(=O)O",
    "benzene":        "c1ccccc1",
    "cholesterol":    "CC(C)CCCC(C)C1CCC2C1(CCC3C2CC=C4C3(CCC(C4)O)C)C",
}
gen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)
fp = lambda m: gen.GetFingerprint(m)
ref_fp = fp(known_active)

def lipinski_ok(m):             # Rule of five (max 1 violation allowed)
    v = [Descriptors.MolWt(m) > 500, Descriptors.MolLogP(m) > 5,
         Lipinski.NumHDonors(m) > 5, Lipinski.NumHAcceptors(m) > 10]
    return sum(v) <= 1

rows = []
for name, smi in library.items():
    m = Chem.MolFromSmiles(smi)
    sim = DataStructs.TanimotoSimilarity(ref_fp, fp(m))
    rows.append((name, sim, Descriptors.MolWt(m), Descriptors.MolLogP(m), lipinski_ok(m)))

print(f"\n{'Compound':16s}{'Tanimoto':>9s}{'MW':>8s}{'logP':>7s}  RO5")
for r in sorted(rows, key=lambda r: -r[1]):
    print(f"{r[0]:16s}{r[1]:9.2f}{r[2]:8.1f}{r[3]:7.2f}  {'pass' if r[4] else 'fail'}")
hits = [r[0] for r in rows if r[1] >= 0.3 and r[4]]
print("\nShortlisted hits (similarity >= 0.3 and passes RO5):", hits)
```

Typical command-line workflow (reference)

```
# Prepare ligand and receptor (Open Babel / AutoDockTools)
obabel ligand.sdf -O ligand.pdbqt --gen3d -p 7.4
obabel receptor_clean.pdb -O receptor.pdbqt -xr

# Config file conf.txt
receptor = receptor.pdbqt
ligand   = ligand.pdbqt
center_x = 10.0   center_y = 12.5   center_z = -3.2
size_x   = 20     size_y   = 20     size_z   = 20
exhaustiveness = 8

# Run docking
vina --config conf.txt --out docked.pdbqt --log log.txt

# Visualise result: open receptor.pdbqt and docked.pdbqt in PyMOL / Chimera
```

Code Explanation

- **run_vina()** builds the Vina command line (receptor, ligand, grid centre/size, exhaustiveness = thoroughness of search, number of modes) and runs it if vina is installed; otherwise it just prints the command so the script is safe to run anywhere. Output file docked.pdbqt contains poses ranked by affinity (kcal/mol).
- **Part B - ligand-based virtual screening** with RDKit: each molecule is written as a SMILES string; Morgan (circular, ECFP-like) fingerprints of 2048 bits are generated; the Tanimoto similarity to the reference active (aspirin) is computed = common bits / total bits.

- **lipinski_ok()** counts violations of the Rule of Five (at most 1 allowed). Compounds with similarity 0.3 or more that pass the filter are shortlisted. Here salicylic acid is the top hit; cholesterol has low similarity and also violates logP.
- For structure-based screening, loop run_vina() over every ligand of the library and sort by the best affinity score; a binding energy lower than about -7 kcal/mol is usually considered promising.

Output (from the tested run)

```
vina not installed - command that would be run:
vina --receptor receptor.pdbqt --ligand ligand.pdbqt --center_x 10.0 --center_y 12.5 --center_z -3.2 --size_x 20
    --size_y 20 --size_z 20 --exhaustiveness 8 --num_modes 9 --out docked.pdbqt

Compound         Tanimoto      MW   logP  RO5
salicylic acid       0.45   138.1   1.09  pass
paracetamol          0.22   151.2   1.35  pass
naproxen             0.20   230.3   3.04  pass
ibuprofen            0.20   206.3   3.07  pass
benzene              0.12    78.1   1.69  pass
caffeine             0.09   194.2  -1.03  pass
cholesterol          0.06   386.7   7.39  pass

Shortlisted hits (similarity >= 0.3 and passes RO5): ['salicylic acid']
```

Conclusion (sample)

Docking set-up for AutoDock Vina was prepared (receptor/ligand preparation, grid box and command), and a virtual screening of a small library was carried out using fingerprint similarity and Lipinski filtering, which shortlisted salicylic acid as the best hit. Docking and virtual screening reduce the number of compounds that must be tested in the laboratory, saving time and cost in drug discovery; the best candidates should be validated by re-docking, MD simulation and wet-lab assays.

Assignment 4: Viva Questions and Answers

Q1. What is molecular docking?

Ans: A computational technique that predicts how a small molecule (ligand) binds to a protein (receptor), giving its binding pose and an estimated binding affinity.

Q2. What is a ligand and what is a receptor?

Ans: The ligand is the small molecule (drug candidate); the receptor/target is the macromolecule, usually a protein, to which it binds.

Q3. What is binding affinity and how is it expressed?

Ans: It measures how strongly a ligand binds. Expressed as Kd, Ki, IC50, or binding free energy dG in kcal/mol; docking scores are more negative for stronger binding.

Q4. Name docking software.

Ans: AutoDock, AutoDock Vina, Glide, GOLD, DOCK, rDock, HADDOCK (protein-protein).

Q5. What is a scoring function?

Ans: A function that estimates binding energy of a pose from terms such as van der Waals, electrostatics, hydrogen bonds and desolvation; used to rank poses and compounds.

Q6. What is a search algorithm in docking?

Ans: The method that explores ligand positions and conformations, e.g. genetic algorithm (AutoDock), Monte-Carlo (Vina), systematic search (Glide).

Q7. What is a grid box?

Ans: A 3D box around the binding site that defines the region where the ligand is allowed to move; defined by centre coordinates and size.

Q8. How do you prepare the protein for docking?

Ans: Remove water and hetero-atoms, add missing atoms/hydrogens, assign charges and atom types, and save as PDBQT.

Q9. How do you prepare the ligand?

Ans: Generate the 3D structure, minimise energy, assign charges, define rotatable bonds and save in PDBQT format.

Q10. What is flexible vs rigid docking?

Ans: In rigid docking both molecules are fixed; in flexible docking the ligand (and sometimes selected receptor side chains) can change conformation. Vina treats the ligand as flexible and the receptor as rigid by default.

Q11. What is virtual screening?

Ans: Computational screening of large compound libraries to identify molecules likely to bind a target, before experimental testing.

Q12. Types of virtual screening?

Ans: Structure-based (docking of each compound into the target) and ligand-based (similarity, pharmacophore, QSAR using known actives).

Q13. What is Lipinski's Rule of Five?

Ans: A drug-likeness guideline: MW at most 500, logP at most 5, H-bond donors at most 5, H-bond acceptors at most 10. More than one violation suggests poor oral bioavailability.

Q14. What is the Tanimoto coefficient?

Ans: A similarity measure between two fingerprints: number of common bits / number of bits present in either. It ranges from 0 (no similarity) to 1 (identical).

Q15. How is a docking protocol validated?

Ans: By re-docking the co-crystallised ligand and checking RMSD below 2 Angstrom from the crystal pose, and by checking enrichment with known actives vs decoys.

Q16. Which file formats are used?

Ans: PDB (structure), PDBQT (docking input with charges and atom types), SDF/MOL2/SMILES (ligands).

Assignment 5: Machine Learning for Genomic Data

Aim / Problem Statement

Apply machine-learning algorithms such as Random Forest and Support Vector Machine to classify genomic data using specific features or markers.

Theory

Genomic data types: genome sequences, annotations, transcriptomics (gene expression), epigenomics (DNA methylation, histone marks) and comparative genomics. In ML terms the data is a matrix of **samples (rows) x features (columns)** - for example genes or SNPs - with a class **label** such as disease / normal.

Challenges: very high dimensionality (thousands of features, few samples - the 'p much greater than n' problem), noise, class imbalance and over-fitting. Remedies: feature selection, scaling, regularisation, cross-validation.

Random Forest: an ensemble of many decision trees trained on bootstrap samples with random feature subsets; the final class is by majority vote. It is robust, handles high-dimensional data, rarely over-fits and gives **feature-importance** scores (which genes matter most).

Support Vector Machine (SVM): finds the hyperplane that separates the classes with the maximum margin; with kernels (RBF, polynomial) it can separate non-linear data. Features must be scaled. Parameters: C (margin softness) and gamma (kernel width).

Neural networks / deep learning (CNNs, RNNs) are used for raw sequence data (e.g. predicting binding sites or variant effects).

Evaluation metrics (from the confusion matrix TP, TN, FP, FN): Accuracy = (TP+TN)/all; Precision = TP/(TP+FP); Recall (sensitivity) = TP/(TP+FN); F1 = 2PR/(P+R), the harmonic mean - useful for imbalanced data. **Cross-validation** (k-fold) gives a more reliable estimate than a single split. ROC-AUC is also common.

Interpretation: feature importance scores, heat-maps/bar plots, and pathway/GO analysis of the top genes to extract biological meaning.

Algorithm / Steps

1. Load the feature matrix and labels (here a simulated gene-expression dataset with 300 samples x 200 genes).
2. Pre-process: stratified train/test split, feature scaling for SVM.
3. Train Random Forest and SVM; predict the test set.
4. Evaluate with accuracy, precision, recall, F1, confusion matrix and 5-fold cross-validation.
5. Extract and plot Random Forest feature importances; relate top genes to biology.

Program (a5_ml_genomics.py)

```
# Assignment 5: Machine Learning for genomic data (Random Forest vs SVM)
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)

# 1. Simulated gene-expression matrix: 300 samples x 200 genes, 15 informative genes
```

```
X, y = make_classification(n_samples=300, n_features=200, n_informative=15,
                           n_redundant=10, weights=[0.6, 0.4], random_state=7)
genes = [f"Gene{i}" for i in range(X.shape[1])]
df = pd.DataFrame(X, columns=genes); df["label"] = y          # 0 = normal, 1 = disease
print("Class counts:", df.label.value_counts().to_dict())
# (real data: df = pd.read_csv("expression.csv"); X = df.drop("label", axis=1))

# 2. Pre-processing / split
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=1)

# 3. Models
models = {
    "Random Forest": RandomForestClassifier(n_estimators=300, random_state=1),
    "SVM (RBF)":     make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1, gamma="scale")),
}
for name, mdl in models.items():
    mdl.fit(Xtr, ytr); p = mdl.predict(Xte)
    cv = cross_val_score(mdl, X, y, cv=5, scoring="f1").mean()
    print(f"\n=== {name} ===")
    print(f"Accuracy {accuracy_score(yte,p):.3f}  Precision {precision_score(yte,p):.3f}  "
          f"Recall {recall_score(yte,p):.3f}  F1 {f1_score(yte,p):.3f}  (5-fold CV F1 {cv:.3f})")
    print("Confusion matrix [[TN FP],[FN TP]]:\n", confusion_matrix(yte, p))

# 4. Feature importance (Random Forest) = candidate biomarker genes
rf = models["Random Forest"]
imp = pd.Series(rf.feature_importances_, index=genes).sort_values(ascending=False)
print("\nTop 10 important genes:\n", imp.head(10).round(4).to_string())
imp.head(15)[::-1].plot.barh(figsize=(6, 4), color="teal")
plt.xlabel("Importance"); plt.title("Top genes (Random Forest)")
plt.tight_layout(); plt.savefig("a5_feature_importance.png", dpi=150)
```

Code Explanation

- **make_classification** creates a synthetic dataset with 15 truly informative genes among 200, and a 60:40 class imbalance. For real data (e.g. TCGA, GEO) load a CSV with pandas and separate X (features) and y (labels).
- **train_test_split(stratify=y)** keeps the same class proportion in training and test sets.
- **make_pipeline(StandardScaler(), SVC(...))** scales features and then trains an RBF-kernel SVM; scaling is mandatory for SVM but not for Random Forest.
- **cross_val_score(cv=5, scoring='f1')** repeats train/test 5 times to give a stable performance estimate.
- **feature_importances_** in the Random Forest ranks genes; those with highest scores are candidate biomarkers and can be sent to pathway analysis. Results: both models reach about 83-85 % accuracy; Random Forest has precision 1.0 but misses some positives (recall about 0.63) - this is why several metrics, not only accuracy, must be reported.

Output (from the tested run)

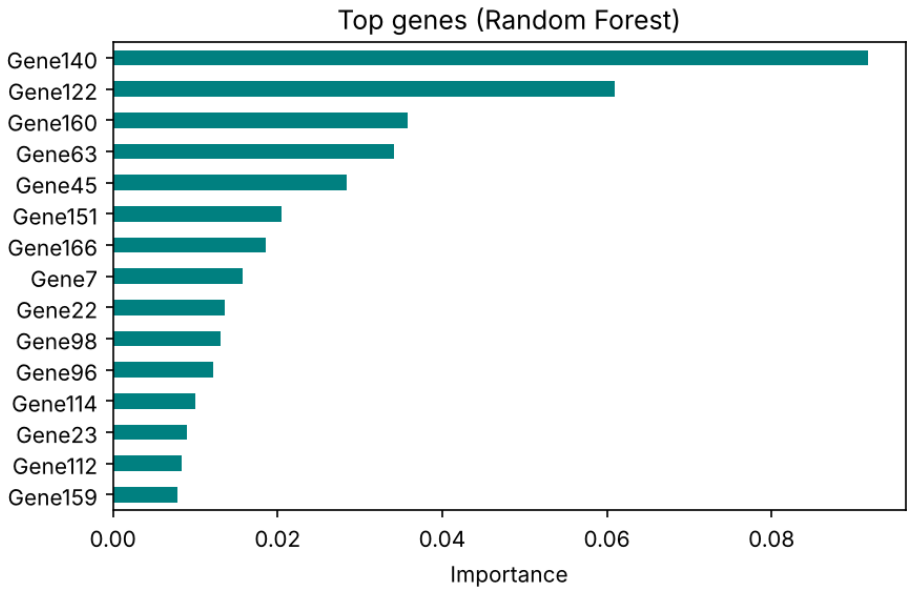
```
Class counts: {0: 179, 1: 121}

=== Random Forest ===
Accuracy 0.853  Precision 1.000  Recall 0.633  F1 0.776  (5-fold CV F1 0.765)
Confusion matrix [[TN FP],[FN TP]]:
 [[45  0]
 [11 19]]

=== SVM (RBF) ===
Accuracy 0.827  Precision 0.947  Recall 0.600  F1 0.735  (5-fold CV F1 0.750)
Confusion matrix [[TN FP],[FN TP]]:
 [[44  1]
 [12 18]]

Top 10 important genes:
 Gene140    0.0917
Gene122    0.0609
Gene160    0.0358
Gene63     0.0341
Gene45     0.0283
Gene151    0.0205
Gene166    0.0186
```

```
Gene7      0.0157
Gene22     0.0136
Gene98     0.0130
```



Top 15 genes ranked by Random Forest feature importance (candidate biomarkers).

Conclusion (sample)

Random Forest and SVM classifiers were trained on genomic (gene-expression) data and evaluated using accuracy, precision, recall, F1-score and cross-validation. Random Forest performed slightly better (accuracy about 0.85, F1 about 0.78) and its feature importance revealed the genes that contribute most to classification. Machine learning therefore helps to classify samples and to discover biologically meaningful markers in genomics research.

Assignment 5: Viva Questions and Answers

Q1. What is a genomic data set in ML terms?

Ans: A feature matrix with samples as rows and genomic features (gene expression, SNPs, methylation) as columns, plus a class label for supervised learning.

Q2. Supervised vs unsupervised learning?

Ans: Supervised uses labelled data to predict a class/value (classification, regression); unsupervised finds structure without labels (clustering, PCA).

Q3. What is a Random Forest?

Ans: An ensemble of decision trees, each trained on a bootstrap sample and random subset of features; predictions are combined by majority voting.

Q4. Why is Random Forest good for genomics?

Ans: It copes with many features and few samples, captures non-linear interactions, is resistant to over-fitting and gives feature importance.

Q5. What is an SVM?

Ans: A classifier that finds the maximum-margin hyperplane separating classes; kernels allow non-linear boundaries.

Q6. What is a kernel and name some?

Ans: A function that maps data to a higher-dimensional space implicitly so non-linear classes become separable: linear, polynomial, RBF (Gaussian), sigmoid.

Q7. Why scale features for SVM?

Ans: SVM relies on distances; features with large ranges would dominate. Scaling (StandardScaler) gives all features equal influence.

Q8. What is over-fitting and how to avoid it?

Ans: The model learns noise in the training data and performs poorly on new data. Avoid with cross-validation, regularisation, feature selection, simpler models, more data.

Q9. What is the 'curse of dimensionality' in genomics?

Ans: Having far more features (genes) than samples makes models prone to over-fitting and distances less meaningful; need feature selection/dimension reduction.

Q10. Define accuracy, precision, recall and F1.

Ans: Accuracy = correct/all; precision = TP/(TP+FP); recall = TP/(TP+FN); F1 = harmonic mean of precision and recall.

Q11. Why is accuracy misleading for imbalanced data?

Ans: A model predicting the majority class always can still have high accuracy. Precision, recall, F1, balanced accuracy or ROC-AUC give a truer picture.

Q12. What is a confusion matrix?

Ans: A table of true vs predicted classes showing TP, TN, FP and FN counts.

Q13. What is k-fold cross-validation?

Ans: Data is split into k parts; the model is trained on k-1 and tested on the remaining one, repeating k times, and the scores are averaged.

Q14. What is feature importance?

Ans: A score showing how much each feature contributes to predictions, e.g. the decrease in impurity in Random Forest trees; used to identify biomarker genes.

Q15. How can you improve the model?

Ans: Hyper-parameter tuning (GridSearchCV), feature selection (e.g. variance filter, LASSO), balancing classes (SMOTE, class weights), more data, ensemble methods.

Assignment 6: Agricultural Genomics and Crop Improvement

Aim / Problem Statement

Analyse crop genomic data to identify genetic markers (SNPs) associated with desirable traits such as disease resistance or yield.

Theory

Agricultural genomics studies the genomes of crops and livestock to understand the genetic basis of traits. Applications: crop improvement (yield, drought tolerance, nutrition), livestock breeding (milk, meat quality), pest/disease resistance and food safety/traceability.

Data types: genome sequencing (reference genome), transcriptomics, proteomics and metagenomics (soil/root/gut microbiomes). Public sources: Ensembl Plants, Gramene, NCBI, Phytozome, 3000 Rice Genomes.

Genetic markers: DNA variants used as signposts for traits - SNPs (single nucleotide polymorphisms), SSRs/microsatellites, indels. SNPs are the most common marker; genotype is coded as 0, 1 or 2 copies of the alternate allele. **Marker-assisted selection (MAS)** uses markers linked to a trait to select plants early in breeding.

Association analysis: tests whether genotype at a variant is correlated with the phenotype. For quantitative traits (yield) a linear regression is used; for case/control (resistant vs susceptible) a chi-square/logistic regression test. The effect size (beta) shows how much each allele changes the trait.

GWAS (Genome-Wide Association Study) repeats the test for every SNP across the genome in many individuals. Because thousands of tests are done, significance is corrected by **Bonferroni** (0.05 / number of SNPs) or FDR. Results are shown in a **Manhattan plot** (-log10 p against genomic position) and a QQ plot (to detect inflation). Real GWAS also corrects for population structure and relatedness (PCA covariates, mixed models - GAPIT, TASSEL, GEMMA, PLINK).

Quality control: remove SNPs with low minor allele frequency (MAF), high missingness or Hardy-Weinberg deviations. **Linkage disequilibrium (LD)** means nearby SNPs are inherited together, so a significant SNP may only be near the causal gene.

Algorithm / Steps

1. Obtain genotype matrix (VCF/PLINK; e.g. produced with BWA + GATK/bcftools) and phenotype data.
2. QC: filter by minor allele frequency.
3. Run association test (linear regression) for each SNP.
4. Correct for multiple testing (Bonferroni).
5. Plot Manhattan plot, list significant markers and annotate nearby genes (BEDTools/Ensembl Plants).

Program (a6_gwas_crops.py)

```
# Assignment 6: Agricultural genomics - simple GWAS for a crop trait (e.g. yield)
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(11)
n_lines, n_snps = 300, 1000                     # 300 crop lines, 1000 SNPs

# 1. Genotypes coded 0/1/2 (copies of alternate allele); chromosome + position
maf = rng.uniform(0.1, 0.5, n_snps)
```

```
G = rng.binomial(2, maf, size=(n_lines, n_snps))
snp = pd.DataFrame({"SNP": [f"rs{i}" for i in range(n_snps)],
                    "CHR": np.repeat(np.arange(1, 11), n_snps // 10),
                    "POS": np.tile(np.arange(n_snps // 10), 10)})

# 2. Phenotype = yield. SNPs 120, 450, 777 are the true causal variants
causal = {120: 2.0, 450: 1.5, 777: -1.8}
y = rng.normal(50, 5, n_lines) + sum(b * G[:, i] for i, b in causal.items())

# 3. QC: remove SNPs with MAF < 5 %
freq = G.mean(axis=0) / 2
keep = (np.minimum(freq, 1 - freq) >= 0.05)
G, snp = G[:, keep], snp[keep].reset_index(drop=True)

# 4. Association test: linear regression of phenotype on each SNP
res = [stats.linregress(G[:, j], y) for j in range(G.shape[1])]
snp["beta"] = [r.slope for r in res]
snp["pvalue"] = [r.pvalue for r in res]
snp["-log10p"] = -np.log10(snp.pvalue)

# 5. Multiple-testing correction
bonf = 0.05 / len(snp)
snp["significant"] = snp.pvalue < bonf
print(f"SNPs tested: {len(snp)}   Bonferroni threshold: {bonf:.2e}")
print(snp.sort_values("pvalue").head(8).round(5).to_string(index=False))
print("\nSignificant markers:", snp.loc[snp.significant, "SNP"].tolist())

# 6. Manhattan plot
snp["x"] = np.arange(len(snp))
plt.figure(figsize=(9, 4))
for chrom, d in snp.groupby("CHR"):
    plt.scatter(d.x, d["-log10p"], s=6, c="#1f77b4" if chrom % 2 else "#7fb3e0")
plt.axhline(-np.log10(bonf), c="red", ls="--", lw=1, label="Bonferroni")
plt.xlabel("SNP index (chromosomes 1-10)"); plt.ylabel("-log10(p)")
plt.title("Manhattan plot - yield GWAS"); plt.legend()
plt.tight_layout(); plt.savefig("a6_manhattan.png", dpi=150)
```

PLINK equivalent for real data (reference)

```
plink --vcf crop.vcf --make-bed --out crop --allow-extra-chr
plink --bfile crop --maf 0.05 --geno 0.1 --make-bed --out crop_qc
plink --bfile crop_qc --pheno yield.txt --assoc --allow-extra-chr --out gwas_yield
# --assoc (quantitative: linear regression) ; add --linear --covar pcs.txt to correct for population structure
```

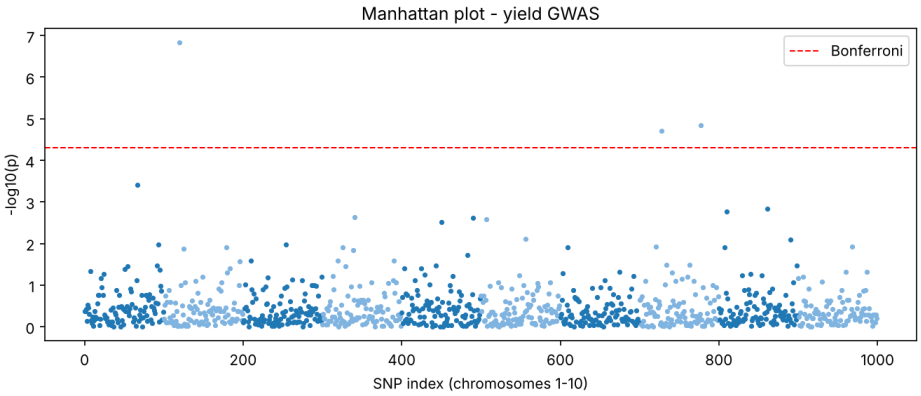
Code Explanation

- **Simulation**: 300 crop lines x 1000 SNPs on 10 chromosomes, genotypes 0/1/2 drawn from allele frequencies. Yield = random background + effects of 3 causal SNPs (rs120, rs450, rs777) so the method can be verified. For real data, load genotypes from a PLINK/VCF-derived table.
- **MAF filter** removes rare variants (frequency below 5 %) that give unreliable statistics.
- **scipy.stats.linregress** regresses yield on genotype for every SNP, giving slope (beta) and p-value.
- **Bonferroni** threshold = 0.05 / 1000 = 5e-05. The test recovered rs120 and rs777 (true causal SNPs) and rs728 - a SNP not simulated as causal (a false positive by chance/LD), a reminder to validate GWAS hits; the causal SNP rs450 was not detected at this threshold (smaller effect, limited power with 300 lines).
- **Manhattan plot**: x = SNP position (alternating colours per chromosome), y = -log10(p); the dashed red line is the significance cut-off.

Output (from the tested run)

```
SNPs tested: 1000   Bonferroni threshold: 5.00e-05
  SNP  CHR  POS     beta  pvalue  -log10p  significant
rs120    2   20  3.45542 0.00000  6.82691         True
rs777    8   77 -1.93882 0.00001  4.84760         True
rs728    8   28  2.43347 0.00002  4.71297         True
 rs67    1   67 -1.70006 0.00040  3.40171        False
rs861    9   61  1.74479 0.00144  2.84244        False
rs810    9   10 -1.43847 0.00172  2.76444        False
rs341    4   41 -1.69982 0.00232  2.63379        False
rs490    5   90 -1.33577 0.00246  2.60855        False
```

```
Significant markers: ['rs120', 'rs728', 'rs777']
```



Manhattan plot: each dot is a SNP; points above the red Bonferroni line are significantly associated with yield.

Conclusion (sample)

Genomic data of crop lines was analysed by GWAS-style association testing. After QC and Bonferroni correction, SNPs significantly associated with yield were identified and displayed in a Manhattan plot, including the true causal markers rs120 and rs777. Such markers can be used for marker-assisted selection and to find candidate genes for crop improvement; real studies must also correct for population structure and validate the markers.

Assignment 6: Viva Questions and Answers

Q1. What is agricultural genomics?

Ans: The study of genomes of crops and livestock to understand and improve traits such as yield, disease resistance, drought tolerance and quality.

Q2. What is a SNP?

Ans: Single Nucleotide Polymorphism - a variation of a single base at a specific genome position among individuals (for example A in one line, G in another).

Q3. How are genotypes coded for association analysis?

Ans: As 0, 1, 2 - the number of copies of the alternate (minor) allele in the individual.

Q4. What is a genetic marker?

Ans: A DNA sequence with a known location that varies among individuals and is used to track inheritance of traits (SNP, SSR, indel).

Q5. What is marker-assisted selection?

Ans: Using markers linked to a desirable gene/QTL to select plants in breeding programmes at an early stage without waiting for the trait to show.

Q6. What is association analysis?

Ans: A statistical test of whether a genetic variant is correlated with a trait by comparing individuals with different genotypes or with and without the trait.

Q7. What is GWAS?

Ans: Genome-Wide Association Study: testing hundreds of thousands of SNPs across the genome in many individuals to find variants associated with a trait.

Q8. Why is multiple testing correction required in GWAS?

Ans: With thousands of tests, many SNPs would look significant by chance. Bonferroni (0.05/n) or FDR controls false positives.

Q9. What is a Manhattan plot?

Ans: A scatter plot of -log10 p-values vs genomic position; tall peaks above the threshold line indicate associated loci. It looks like a city skyline.

Q10. What is a QQ plot used for?

Ans: To compare observed p-values with expected ones under the null; deviation of the whole line indicates inflation (population structure), deviation only at the tail indicates real signal.

Q11. What is MAF and why filter on it?

Ans: Minor allele frequency - the frequency of the less common allele. Rare variants have low statistical power and more genotyping errors, so SNPs with MAF below about 5 % are removed.

Q12. What is linkage disequilibrium?

Ans: Non-random association of alleles at nearby loci; SNPs close together are inherited together, so an associated SNP may only tag the true causal variant.

Q13. What is population structure and how is it handled?

Ans: Systematic ancestry differences among samples cause spurious associations. Handled with PCA covariates or mixed linear models (kinship matrix), e.g. GAPIT, TASSEL, GEMMA.

Q14. What is a QTL?

Ans: Quantitative Trait Locus - a genomic region containing genes that influence a quantitative trait like yield or height.

Q15. Which tools/formats are used?

Ans: BWA/Bowtie2 for alignment, SAMtools/BEDTools for processing, GATK/bcftools for variant calling (VCF), PLINK/TASSEL/GAPIT for GWAS, Python/R for analysis.

Quick Revision: Formulas and Key Terms

Term	Formula / Meaning
GC content	(G + C) / (A + T + G + C) x 100
Stop codons / start codon	TAA, TAG, TGA / ATG
Reading frames	3 per strand x 2 strands = 6
% Identity	identical aligned residues / alignment length x 100
log2 fold change	log2(treated / control)
Benjamini-Hochberg	padj(i) = p(i) x n / rank(i), made monotonic
Binding energy (Vina)	kcal/mol; more negative = stronger binding
Tanimoto	common bits / (bits in A + bits in B - common bits)
Lipinski rule of 5	MW <= 500, logP <= 5, HBD <= 5, HBA <= 10
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Precision / Recall	TP / (TP + FP) / TP / (TP + FN)
F1-score	2 x P x R / (P + R)
Bonferroni threshold	0.05 / number of tests
RMSD	sqrt(sum of squared atom distances / N); below 2 Angstrom = good